

Tipos de datos enumerados: enum

Piensen en los enums como si fueran una lista cerrada de opciones. Imaginen que están organizando un juego y solo pueden elegir entre "Piedra", "Papel" y "Tijera". No hay más opciones. Los enums nos permiten definir este tipo de colecciones finitas y con nombre,

evitando errores y haciendo nuestro código mucho más claro y seguro.



Concepto: ¿Qué es un Enum?

Un **enum** es un tipo de dato especial que nos permite definir una **colección de constantes** con nombre. Piénsalo como una clase, pero con un propósito muy específico: agrupar un conjunto de valores fijos y relacionados. En lugar de usar números mágicos (int 1 para "rojo", int 2 para "verde"), usamos nombres descriptivos (Color.ROJO, Color.VERDE). Esto hace que el código sea mucho más legible, fácil de mantener y menos propenso a errores.

Aplicación: ¿Cuándo usar un Enum?

Los enums son perfectos para situaciones donde tienes un conjunto de valores que no va a cambiar. Algunas de las aplicaciones más comunes incluyen:

- **Días de la semana:** LUNES, MARTES, MIÉRCOLES, etc.
- **Estados de un objeto:** PENDIENTE, EN_PROCESO, COMPLETADO.
- **Puntos cardinales:** NORTE, SUR, ESTE, OESTE.
- **Tipos de cartas:** CORAZONES, DIAMANTES, TRÉBOLES, PICAS.

- **Tipos de vehículos:** AUTOMOVIL, MOTOCICLETA, BICICLETA.

Usar enums en estos casos garantiza que solo se utilicen los valores permitidos, evitando que, por ejemplo, alguien ingrese un día de la semana que no existe.

Ejemplo: El Semáforo

Vamos a crear un ejemplo simple y visual. Imaginemos que estamos programando el comportamiento de un semáforo.

Java

```
public enum EstadoSemaforo {  
    ROJO("Detenerse"),  
    AMARILLO("Precaución"),  
    VERDE("Avanzar");  
  
    private final String accion;  
  
    EstadoSemaforo(String accion) {  
        this.accion = accion;  
    }  
  
    public String getAccion() {  
        return accion;  
    }  
}
```

En este código:

- Definimos un enum llamado `EstadoSemaforo`.
- Cada constante (`ROJO`, `AMARILLO`, `VERDE`) es un objeto `EstadoSemaforo`.
- A cada constante le asociamos un valor ("`Detenerse`") a través de un constructor. Los enums pueden tener constructores, métodos y atributos.

- El método `getAccion()` nos permite acceder a la descripción de cada estado.

Ahora, ¿cómo lo usamos?

```
public class Trafico {  
  
    public static void main(String[] args) {  
        EstadoSemaforo estadoActual = EstadoSemaforo.VERDE;  
        System.out.println("El semáforo está en " + estadoActual);  
        System.out.println("La acción a tomar es: " +  
estadoActual.getAccion());  
  
        // Cambiamos el estado  
        estadoActual = EstadoSemaforo.AMARILLO;  
        System.out.println("\nAhora el semáforo está en " +  
estadoActual);  
        System.out.println("La acción a tomar es: " +  
estadoActual.getAccion());  
    }  
}
```

Salida:

```
El semáforo está en VERDE  
La acción a tomar es: Avanzar  
  
Ahora el semáforo está en AMARILLO  
La acción a tomar es: Precaución
```

CLASES INMUTABLES.

Piensen en una clase inmutable como un libro de texto antiguo en una biblioteca pública. Una vez que se publica y se pone en la estantería, **nadie puede cambiar su contenido.**

Si alguien necesita una versión con anotaciones o correcciones, tiene que hacer una copia completa y crear un nuevo libro. Las clases inmutables funcionan de la misma manera: una vez que un objeto se crea, su estado no puede ser modificado.

Concepto: ¿Qué es una clase inmutable?

Una **clase inmutable** es una clase cuyos objetos, una vez creados, no pueden cambiar su estado. Esto significa que los valores de sus campos (sus atributos) no pueden ser modificados después de la instanciación. La inmutabilidad es una propiedad fundamental que trae consigo grandes ventajas, especialmente en entornos de programación concurrente, donde múltiples hilos de ejecución acceden a los mismos datos.

Aplicación: ¿Cuándo usar clases inmutables?

Las clases inmutables son ideales para representar datos que no deberían cambiar. Esto incluye:

- **Objetos de valor:** Datos como fechas (`LocalDate`), colores (`Color`), o puntos en un plano (`Point`). No tiene sentido "cambiar" el valor de una fecha, sino más bien crear una nueva fecha.
- **Datos compartidos:** Cuando varios hilos o partes de un programa acceden a la misma información, la inmutabilidad garantiza que nadie la pueda modificar accidentalmente, eliminando los problemas de sincronización y los errores por estado inconsistente.

- **Claves en colecciones:** Objetos inmutables son perfectos para ser usados como llaves en colecciones como `HashMap` o `HashSet`, ya que su valor hash (el código que los identifica) no cambiará, asegurando que la búsqueda funcione correctamente.
-

Ejemplo en Java: Un Punto en el Plano

Vamos a crear una clase `Punto` que sea inmutable. Para lograrlo, debemos seguir una serie de reglas estrictas:

1. **Declara la clase como `final`:** Esto evita que la clase pueda ser extendida (heredada), garantizando que su comportamiento no sea modificado.
2. **Haz que todos los campos sean `final` y `private`:** Los campos `final` solo pueden ser inicializados una vez, ya sea en la declaración o en el constructor. `private` los protege del acceso directo desde fuera de la clase.
3. **No ofrezcas métodos "setter":** No debe haber ningún método público que permita cambiar los valores de los campos.
4. **Si algún campo es mutable, crea una copia:** Si tu clase inmutable contiene un objeto mutable (como una `ArrayList`), debes crear una copia defensiva en el constructor y devolver una copia en el *getter* para evitar que se modifique el estado interno.

Aquí está el código de nuestra clase `Punto`:

```
public final class Punto {  
    private final int x;  
    private final int y;  
}
```

```

public Punto(int x, int y) {
    this.x = x;
    this.y = y;
}

public int getX() {
    return x;
}

public int getY() {
    return y;
}

public Punto desplazar(int dx, int dy) {
    // En lugar de modificar el objeto, creamos uno nuevo
    return new Punto(this.x + dx, this.y + dy);
}

@Override
public String toString() {
    return "Punto(" + x + ", " + y + ")";
}
}

```

En este ejemplo, la clase `Punto` es `final` y sus campos `x` e `y` son `private final`.

No hay *setters*. El método `desplazar()` no cambia el objeto actual, sino que **devuelve un nuevo objeto** `Punto` con las coordenadas modificadas.

Así es como lo usaríamos:

```

public class EjemploInmutabilidad {
    public static void main(String[] args) {
        Punto p1 = new Punto(5, 10);
        System.out.println("Punto original: " + p1); // Salida: Punto(5,
10)

        // Intentar desplazar el punto
    }
}

```

```
Punto p2 = p1.desplazar(2, 3);  
System.out.println("Punto original después del desplazamiento: "  
+ p1); // Salida: Punto(5, 10)  
System.out.println("Nuevo punto desplazado: " + p2); // Salida:  
Punto(7, 13)  
  
// ¡El objeto p1 NO cambió! Se creó un nuevo objeto p2.  
}  
}
```